Chapter 74

Using Modifiers, Expressions, and Custom Controls

Some of the most powerful aspects of Fusion are the different ways it allows you to go beyond the standard tools delivered with the application.

This chapter provides an introduction to a variety of advanced features, including Modifiers, Expressions, and Scripting, which can help you extend the functionality and better integrate Fusion into your studio.

Contents

The Contextual Menu for Parameters in the Inspector

Most of the features in this chapter are accessed via a contextual menu that appears when you right-click most parameters in the Inspector. Different contextual menus are available based on where in the Inspector you right-click. Specifically, right-clicking over parameter names or sliders displays a feature-rich contextual menu that can add animation, expression fields, modifiers to extend functionality, as well as publishing and linking capabilities, allowing you to adjust multiple controls simultaneously.

Using Modifiers

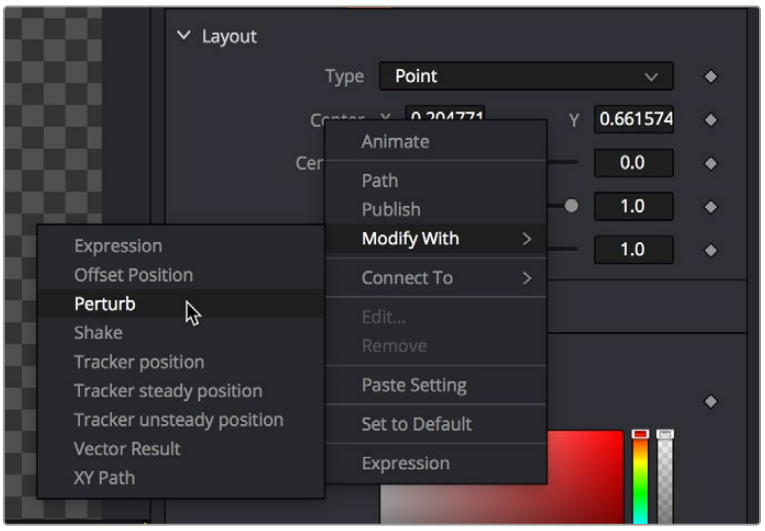
Parameters can be controlled with modifiers, which are extensions to a node's toolset. Many modifiers can automatically create animation that would be difficult to achieve manually. Modifiers can be as simple as keyframe animation or linking the parameters to other nodes, or modifiers can be complex expressions, procedural functions, external data, third-party plugins, or fuses.

Adding the Right Modifier for the Job

Which modifiers are available depends on the type of parameter you're right-clicking over. Numeric values, text, polylines, gradients, and points each have different sets of modifiers that will work with them, so the Modify With menu is filtered based on each parameter.

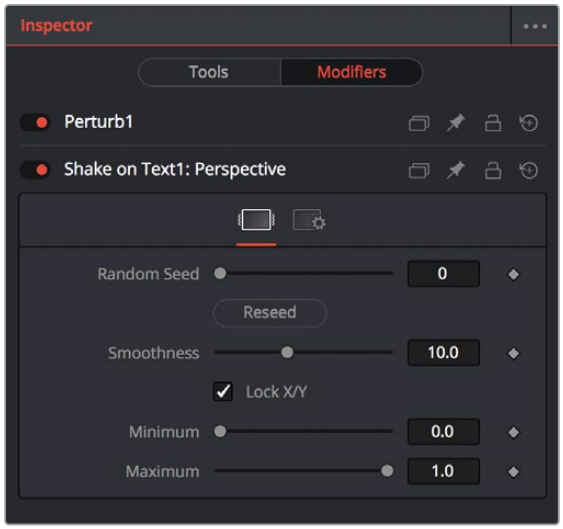
Adding Modifiers to Individual Parameters

You add modifiers to a parameter using the Inspector's contextual menu, or by right-clicking the onscreen control for a parameter in the viewer. Either way, a dynamic list of modifiers that are appropriate for the selected parameters is displayed. For instance, a Perturb modifier can be added to any slider to auto-animate the parameter by randomly wiggling the value. Once the modifier is added, controls are displayed in the Modifiers tab to adjust the speed and amplitude of the random animation.



The Inspector contextual menu is used to add a Perturb modifier to the Center X and Y parameters

A modifier's controls are displayed in the Modifiers tab of the Inspector. When a selected node has a modifier applied, the Modifiers tab will become highlighted as an indication. The tab remains grayed out if no modifier is applied.



The Modifiers tab with two modifiers applied

Modifiers appear with header bars and header controls just like the tools for nodes. A modifier's title bar can also be dragged into a viewer to see its output.

Combining Modifiers and Keyframes

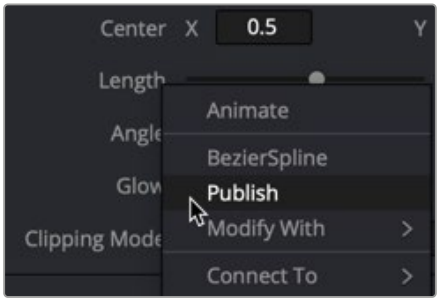
Modifiers that auto-animate parameters like Perturb and Shake can be combined with keyframes to create more natural, organic looking animations. For instance, you can create the general motion path for an element by keyframing the Center X and Y parameters, and then apply a modifier to the same parameters to create a secondary wiggling motion.

To combine a keyframed motion path with a Perturb modifier:

1. Add a Transform to an image like a butterfly or spaceship.
2. Select the Transform node in the Node Editor.
3. In the Inspector, click the Keyframe button to the right of the Center X and Y parameters.
4. Position the butterfly or spaceship image where you want the start of the animation to begin.
5. Continue to move the playhead in the render range and reposition the image until you create a figure-8 motion path.
6. Right-click over the Center X label in the Inspector and choose Modify With > Perturb.
7. Click the Modifiers tab at the top of the Inspector and adjust random, wiggling motion by setting the Strength, Wobble, and Speed parameters while the animation plays.

Publishing a Parameter

The Publish modifier makes the value of a parameter available, so that other parameters can connect to it. This allows you to simultaneously use one slider to adjust other parameters on the same or different nodes. For instance, publishing a motion path allows you to connect multiple objects to the same path.



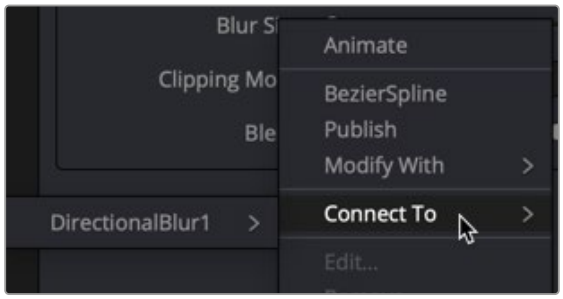
Publish a parameter in order to link other parameters to it

Once a parameter is published, you can right-click another parameter and choose Connect To > [published parameter name] from the contextual menu. The two values are linked, and changing the parameter value of one in the Modifiers tab changes the other.

Using the pick whip between two parameters provides similar linking behavior with more flexibility. Pick whipping between parameters is covered later in this chapter.

Connecting Multiple Parameters to One Modifier

A single modifier or published parameter can be applied to multiple parameters so they all act as one. This is handled through the Connect To submenu. As with modifier assignment, the list is filtered to show only options that are suitable for the parameter you've right-clicked on. When you do this, the connection is bidirectional; editing either parameter will cause the other parameter to change.



The Connect To menu connects modifiers to multiple parameters

Adding and Inserting Multiple Modifiers

Modifiers can be connected to each other and branched, just like nodes in the Node Editor. For example, the Calculation modifier outputs a number, but has two Number parameters, both of which can have modifiers added to them. If you want to insert a modifier between the existing modifier and the modified parameter, use the Insert submenu of the parameter's contextual menu.

Available Modifiers in Fusion:

- **Anim Curves:** Adds an animation curves modifier that allows you to dynamically adjust the timing, scaling, and acceleration of an animation.
- **Bézier Spline:** Adds a Bézier spline to the Spline Editor for animating the selected parameter.
- **B-Spline Spline:** Adds a B-Spline spline to the Spline Editor for animating the selected parameter.
- **Calculation:** Creates an indirect link that includes a mathematical expression between two parameters.

- **CoordTransform Position:** Calculates the current 3D position of a given object even after multiple 3D transforms have repositioned the object through the node tree hierarchy.
- **Cubic Spline:** Adds a Cubic spline to the Spline Editor for animating the selected parameter.
- **Expression:** Allows you to add a variable or a mathematical calculation to a parameter, rather than a straight numeric value. The Expression modifier provides controls in the Modifiers tab, giving you more room and parameters than the SimpleExpression
- **From Image:** This modifier takes color samples of an image along a user-definable line and creates a gradient from those samples.
- **Gradient Color Modifier:** Creates a customized gradient and maps it into a specified time range to animate a value.
- **KeyStretcher:** Used to stretch keyframes in a Fusion Title template when trimming the template in the Edit page or Cut page timeline.
- **MIDI Extractor:** Modifies the value of a parameter using the values stored in a MIDI file.
- **Natural Cubic Spline:** Adds a Natural Cubic spline to the Spline Editor for animating the selected parameter.
- **Offset (Angle, Distance, Position):** The three Offset modifiers are used to create variances, or offsets, between two positional values. For instance, when this modifier is added to a size parameter, you can change the size of an object using the distance between two onscreen controls (position and offset).
- **Path:** Produces two splines to control the animation of of an object: An onscreen motion path (spacial) and a Time spline visible in the Spline Editor (temporal).
- **Perturb:** Generates smoothly varying random animation for a given parameter.
- **Probe:** Auto-animates a parameter by sampling the color or luminosity of a specific pixel or rectangular region of an image.
- **Publish:** The first step in linking two non-animated parameters is to use the Publish modifier to publish a parameter. That allows other parameters to use the Connect To submenu and link to the published parameter.
- **Resolve Parameter:** Allows you to modify the duration of a Fusion transition template from the Edit page Timeline. The Resolve Parameter Modifier is applied to any animated parameter instead of keyframing the transition.
- **Shake:** Similar to Perturb, Shake generates smoothly varying random animation for a given parameter.
- **Track:** Attaches a single point tracker to the selected parameter. The tracker can then track an object onscreen to animate the parameter. This is quicker and more direct than using the normal Tracker node; however, it offers less flexibility since the resulting tracker is only a single point and can only be used for the selected parameter.
- **Vector Result:** Similar to the Offset modifier, Vector Result is used to offset position parameters using origin, distance, and angle controls to create a vector. This vector can then be used to adjust any other parameter.
- **XY Path:** Produces an X and Y spline in the Spline Editor to animate the position of an object..

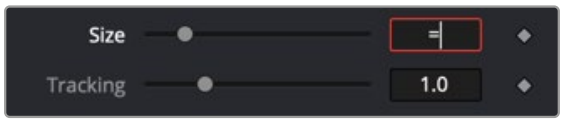
For more information on all modifiers available in Fusion, *see Chapter 124, "Modifiers,"* in the DaVinci Resolve Reference Manual and Chapter 64 in the Fusion Reference Manual.

Performing Calculations in Parameter Fields

You can enter simple mathematical equations directly in a number field to calculate a desired value. For example, typing 2.0 + 4.0 in most number fields will result in a value of 6.0. This can be helpful when you want a parameter to be the sum of two other parameters or a fraction of the screen resolution.

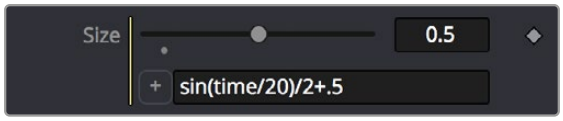
Using SimpleExpressions

Simple Expressions are a special type of script that can be placed alongside the parameter it is controlling. These are useful for setting simple calculations, building unidirectional parameter connections, or a combination of both. You add a SimpleExpression by entering an equals sign directly in the number field of the parameter and then pressing Return.



Entering an equals sign opens the SimpleExpression field with Pick Whip control

An empty field will appear below the parameter, and a yellow indicator will appear to the left. The current value of the parameter will be entered into the number field. Using Simple Expressions, you can enter a mathematical formula that drives the value of a parameter or even links two different parameters. This helps when you want to create an animation that is too difficult or impossible to set up with keyframing. For instance, to create a pulsating object, you can use the sine and time functions on a Size parameter. Dividing the time function can slow down the pulsing while multiplying it can increase the rate.



A SimpleExpression using the sine and time functions

Inside the SimpleExpression text box, you can enter one-line scripts in Lua with some Fusion-specific shorthand. Some examples of Simple Expressions and their syntax include:

Expression	Description
`time`	This returns the current frame number.
`Merge1.Blend`	This returns the value of another input, Blend, from another node, Merge1.
`Merge1:GetValue("Blend", time-5)`	This returns the value from another input, but sampled at a different frame, in this case five frames before the current one.
`sin(time/20)/2+.5`	This returns a sine wave between 0 and 1.

Expression	Description
`iif(Merge1.Blend == 0, 0, 1)`	This returns 0 if the Blend value is 0, and returns 1 if it is not. The iff() function is a shorthand conditional statement, if-then-else.
`iif(Input.Metadata.ColorSpaceID == "sRGB", 0, 1)`	This returns 0 if the image connected to the current node's Input is tagged with the sRGB colorspace. When no other node name is supplied, the expression assumes the Input is coming from the current node. It is equivalent to self.Input. The Input in most, but not all, Fusion nodes is the main image input shown in the Node Editor as an orange triangle. Images have members that you can read, such as Depth, Width, Metadata, and so on.
`Point(Text1.Center.X, Text1.Center.Y-.1)`	Unlike the previous examples, this returns a Point, not a Number. Point inputs use two members, X and Y. In this example, the Point returned is 1/10 of the image height below the Text1's Center. This can be useful for making unidirectional parameter links, like offsetting one Text from another.
`Text1.Center - Point(0,.1)`	This is similar to the previous expression. This SimpleExpression returns Text instead of a Number or Point.
`Text("Colorspace: "..(Merge1.Background.Metadata.ColorSpaceID)`	The string inside the quotes is concatenated with the metadata string, perhaps returning: Colorspace: sRGB
`Text("Rendered "..os.date("%b %d, %Y").. " at "..os.date("%H:%M").."\n on the computer "..os.getenv("COMPUTERNAME").. " running "..os. getenv("OS").."\n from the comp "..ToUNC(comp.Filename))`	This returns a much larger Text, perhaps something like: Rendered Nov 12, 2019 at 15:43 on the computer Rn309 running Windows_NT from the comp \\ SRVR\Proj\Am109\SlateGenerator_A01.comp
`os.date("%H:%M")`	The OS library can pull various information about the computer. In the previous example, os.date gets the date and time in hours:minutes.
`"..os.getenv("COMPUTERNAME").. " running "..os.`	Any environment variable can be read by os.getenv, in this case the computer name and the operating system.
`"\n from the comp "..ToUNC(comp.Filename)`	To get a new line in the Text, \n is used. Various attributes from the comp can be accessed with the comp variable, like the filename, expressed as a UNC path.

TIP: When working with long SimpleExpressions, it may be helpful to drag the Inspector panel out to make it wider or to copy/paste from a text editor or the Console.

After setting an expression that generates animation, you can open the Spline Editor to view the values plotted out over time. This is a good way to check how your SimpleExpression evaluates over time.

A sine wave in the Spline Editor, generated by the expression used for Text1: Size

For more information about writing Simple Expressions, see the Fusion Studio Scripting Guide, and the official Lua documentation.

Pick Whipping to Create an Expression

With a SimpleExpression field open, a + button is displayed on the left side. Dragging the + button onto another control, or "pick whipping," links the two parameters, similar to the Connect To menu. However, unlike a Connect To parameter link, the pick whip allows you to modify the connection by further editing the expression.

Pick whipping to connect one parameter to another quickly

SimpleExpressions can also be created and edited within the Spline Editor. Right-click on the parameter in the Spline Editor and select Set SimpleExpression from the contextual menu. The SimpleExpression will be plotted in the Spline Editor, allowing you to see the result over time.

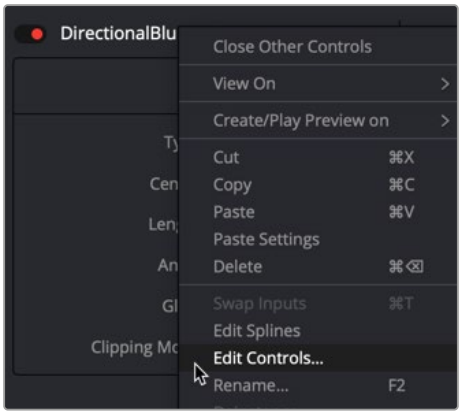
Removing SimpleExpressions

To remove a SimpleExpression, right-click the name of the parameter, and choose Remove Expression from the contextual menu.

Customizing User Controls

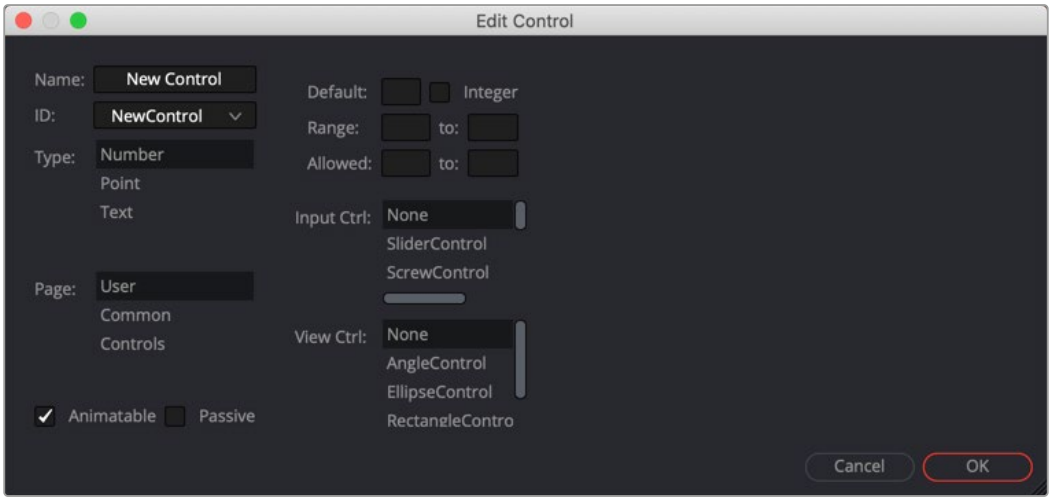
Each tool's parameters are organized in a logical order in the Inspector. The most used controls are closer to the top, and the more subtle refinement controls are lower in the list. Sometimes, though, you may want to add, hide, or change the controls. You often need to do this for SimpleExpressions and macros, but you may also do this for usability and aesthetic reasons for favorites and presets.

Custom controls can be added or edited via the Edit Control dialog, which you access by right-clicking over the node's name in the Inspector and choosing Edit Controls from the menu.



Choose Edit Controls to create and customize parameters

In the Edit Control dialog, you use the ID menu to select an existing parameter or create a new one. You can name the control and define whether it is a text field, number field, or a point using the Type attributes list.

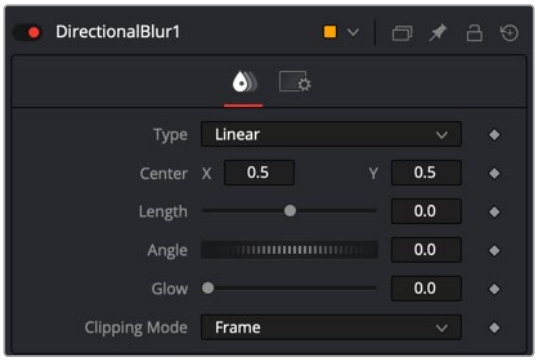


Edit Control dialog

You use the page list to assign the new control to one of the tabs in the Inspector. There are also settings to determine the defaults and ranges, and whether it has an onscreen preview control. The Input Ctrl box contains settings specific to the selected Type, and the View Ctrl attributes box contains a list of onscreen preview controls to be displayed, if any.

All changes made using the Edit Controls dialog get stored in the current tool instance, so they can be copied/ pasted to other nodes in the comp. However, to keep these changes for other comps, you must save the node settings, and add them to the Bins in Fusion Studio or to your favorites.

As an example, we'll customize the controls for a DirectionalBlur:



The Inspector for a default direction blur

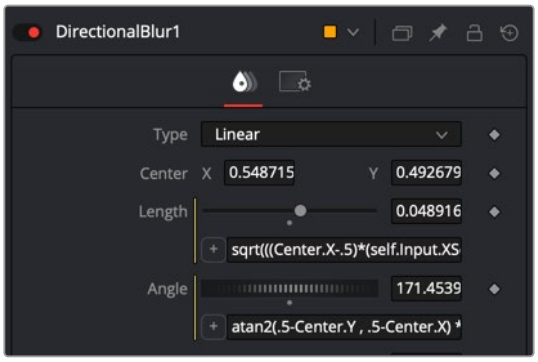
Let's say we wanted a more interactive way of controlling a linear blur in the viewer, rather than using the Length and Angle sliders in the Inspector. Using a SimpleExpression, we'll control the length and angle parameters with the Center parameter's onscreen control in the viewer. The SimpleExpression would look something like this:

For Length:

```
sqrt(((Center.X-.5)*(self.Input.XScale))^2+((Center.Y-.5)*(self.Input.YScale)*(self.Input.Height/self.Input.Width))^2)
```

For Angle:

```
atan2(.5-Center.Y , .5-Center.X) * 180 / pi
```

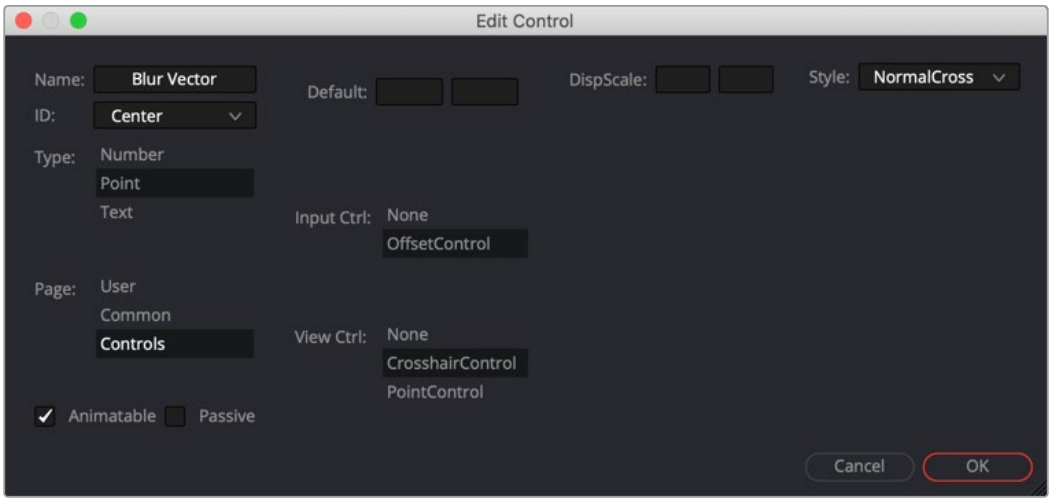


DirectionalBlur controlled by the Center's position.

This admittedly somewhat advanced function does the job fine. Dragging the onscreen control adjusts the angle and length for the directional blur. However, now the names of the parameters are confusing. The Center parameter doesn't function as the center anymore. It is the direction and length of the blur. It should be named "Blur Vector" instead. You no longer need to edit the Length and Angle controls, so they should be hidden away, and since this is only for a linear blur, we don't need the Type menu to include Radial or Zoom. We only need to choose between Linear and Centered. These changes can easily be made in the Edit Controls dialog.

To change the Inspector parameters for the example above:

1. In the Edit Control dialog, select the Center from the ID list.
2. A dialog will appear asking if you would like to Replace, Hide, or Change ID. We'll choose Replace.
3. Change the Name to Blur Vector.
4. Set the Type to Point.
5. Select Controls in the Page list. (Controls is the first tab in the Inspector, normally.)
6. Click OK.



DirectionalBlur Center parameter name changed to Blur Vector.

The new Blur Vector parameter now appears in the Inspector. The internal ID of the control is still Center, so our SimpleExpressions did not change.

To hide the Length and Angle parameters in the Inspector:

1. In the Edit Control dialog, select the Length from the ID list.
2. Select Controls from the Page list.
3. In the input Ctrl list, select Node.
4. Click OK.
5. In the Edit Control dialog, select the Angle from the ID list.
6. In the input Ctrl list, select Node.
7. Click OK.

Finally, to remove Radial and Zoom options from the Type menu:

1. In the Edit Control dialog, select the Type from the ID list.
2. Select Controls from the Page list.
3. Select Radial from the Items list and click Del to remove it.
4. Select Zoom from the Items list and click Del to remove it.
5. Click OK.

The Type menu now includes only two options.

If you want to replace the Type menu with a new checkbox control, you can do that by creating a new control and a very short expression.

To create a new control:

1. In the Edit Control dialog, enter Center Blur in the Name field.
2. Select the New Control from the ID list.
3. Set the Type to Number, and set the Page to Controls.
4. Set the Input Ctrl to CheckboxControl.
5. Click OK.

To make this new checkbox affect the original Type menu, you'll need to add a SimpleExpression to the Type:

```
iif(TypeNew==0, 0, 2)
```

The "iif" operator is known as a conditional expression in Lua script. It evaluates something based on a condition being true or false.

FusionScript

Scripting is an essential means of increasing productivity. Scripts can create new capabilities or automate repetitive tasks, especially those specific to your projects and workflows. Inside Fusion, scripts can rearrange nodes in a comp, manage caches, and generate multiple output files for delivery. They can connect Fusion with other apps to log artist time, send emails, or update webpages and databases.

FusionScript is the blanket term for the scripting environment in Fusion. It includes support for Lua as well as Python 2 and 3 for some contexts. FusionScript also includes libraries to make certain common tasks easier to do with Lua and Python within Fusion.

You can run interactive scripts in various situations. Common scripts include:

- Utility Scripts, using the Fusion application context, are found under the File > Scripts menu.
- Comp Scripts, using the composition context, are found under the Script menu or entered into the Console.
- Tool Scripts, using the tool context, are found in the Tool's context menu > Scripts.

Other script types are available as well, such as Startup Scripts, Scriptlibs, Bin Scripts, Event Suites, Hotkey Scripts, Intool Scripts, and SimpleExpressions. Fusion Studio allows external and command-line scripting as well and network rendering Job and Render node scripting.

FusionScript also forms the basis for Fuses and ViewShaders, which are special scripting-based plugins for tools and viewers that can be used in both Fusion and Fusion Studio.

For more information about scripting, see the Fusion Scripting Documentation, accessible from the Documentation submenu of the Help menu.